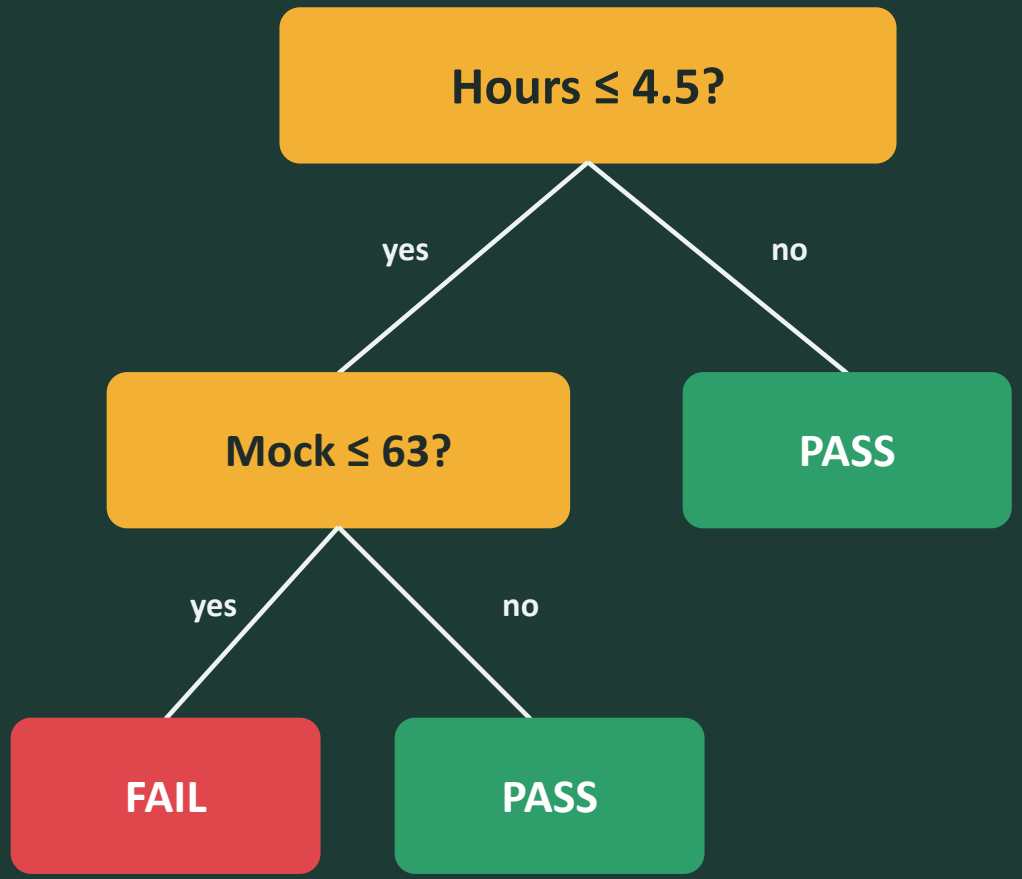
Decision Trees

Teaching a machine to ask good questions



Practical Machine Learning

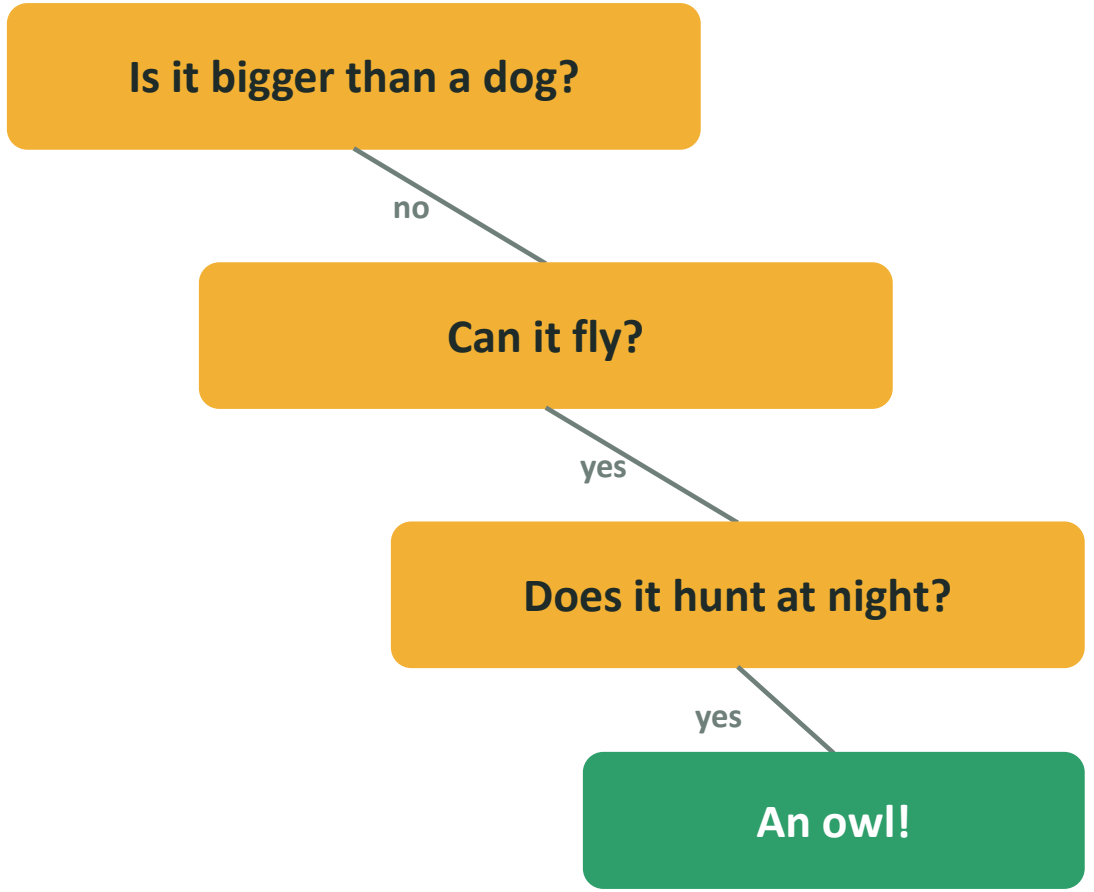
Let’s play 20 Questions

I’m thinking of an animal. You may only ask yes/no questions.

Every answer rules out some possibilities.

Good players ask the question that splits what’s left most cleanly, and they ask it first.

A decision tree is a model that learns which questions to ask, and in what order, from examples.



Meet Riya. Will she pass?

Her exam is next month. Can we predict the result from data, without writing the rule ourselves?

R

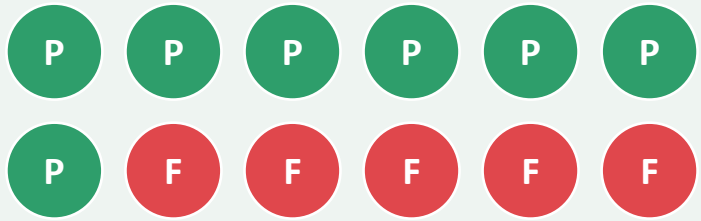
Riya

Studies 4 hours a day

Mock-test score: 70

What we have

12 past students: their study hours, their mock score, and whether they passed.



P = passed, F = failed

Quick poll

If you could ask only one yes/no question about Riya, what would it be?

Write it down. We'll see whether the computer agrees.

Our route today

Five stops. Each one answers the question the previous stop leaves open.

1

The idea

What is a decision tree?

2

Measure

How do we score a question?

3

Grow

How does the tree build itself?

4

Stop

When should it stop asking?

5

Code

The same thing in scikit-learn

Riya gets her answer here

Stop 1 of 5

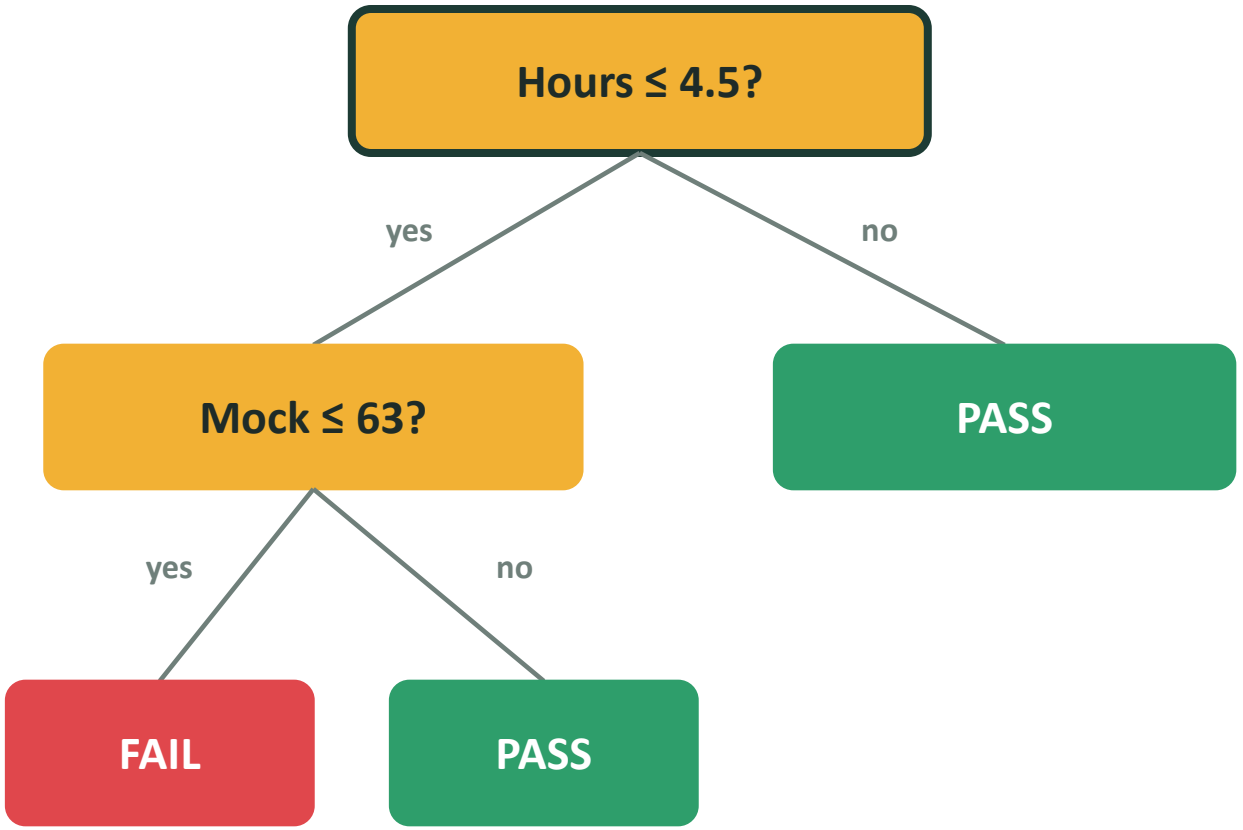
What is a decision tree?

1

A flowchart of yes/no questions that a computer learns from examples.

A decision tree is a learned flowchart

Start at the top. Answer each question, follow the branch, stop at a leaf.



Root node

The first question. Every student starts here.

Decision node

Any question. Only students who reach it are asked.

Branch

An answer. Yes goes left, no goes right.

Leaf

No more questions. The leaf gives the prediction.

You don’t write the rules. The tree does.

That’s the difference between an ordinary program and a machine-learning model.

An ordinary program

```
if hours > 4 and mock > 60:
    result = "Pass"
```

You invent the rule. Why 4 hours? Why 60 marks? It’s your guess.

A decision tree



12 labelled examples

`fit()` ↓

Hours ≤ 4.5?

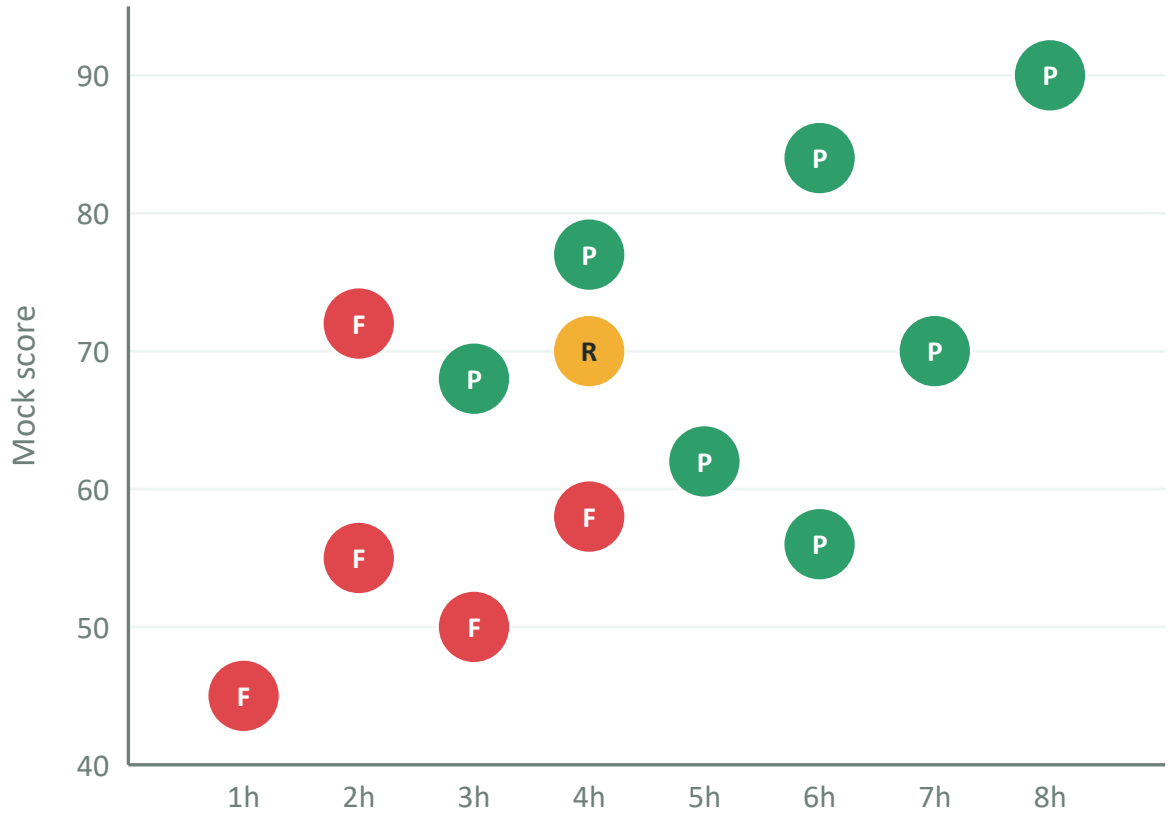
It picks the feature, the threshold and the order of questions by itself.

Big question: how does the tree decide which question is best? First, let’s look at the data.

Our 12 students

Two inputs (hours, mock score) and one known answer (result) for each student.

ID	Hours	Mock	Result
S01	1	45	Fail
S02	2	55	Fail
S03	2	72	Fail
S04	3	50	Fail
S05	3	68	Pass
S06	4	58	Fail
S07	4	77	Pass
S08	5	62	Pass
S09	6	56	Pass
S10	6	84	Pass
S11	7	70	Pass
S12	8	90	Pass



P = passed, F = failed, R = Riya (result unknown)

Stop 2 of 5

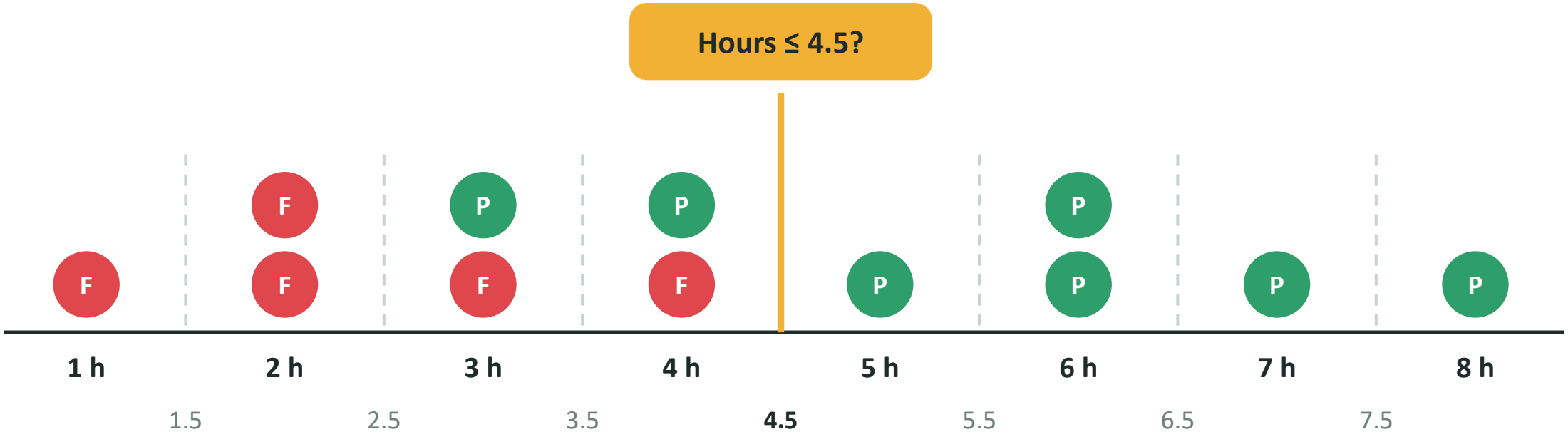
How do we score a question?

2

The tree tries every possible question and keeps the one that makes the cleanest groups.

Where can we cut?

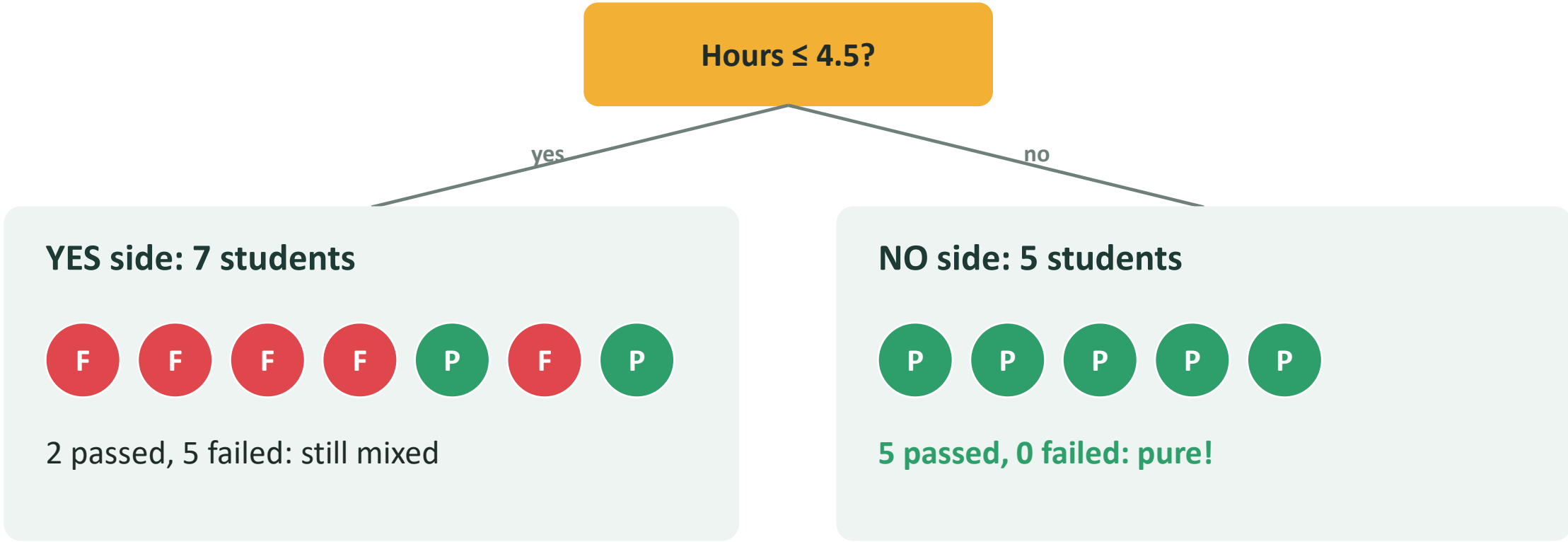
For a number column, the tree tries a cut halfway between every pair of neighbouring values.



Hours has 8 different values, so 7 possible cuts. Mock score has 12 different values, so 11 cuts. That’s 18 candidate questions, and the tree scores every one.

Try one question: Hours ≤ 4.5?

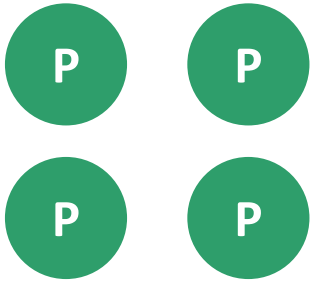
Every student goes to exactly one side. Nobody is lost or counted twice.



To compare questions fairly, we need a number for how mixed a group is.

Pure groups are what we want

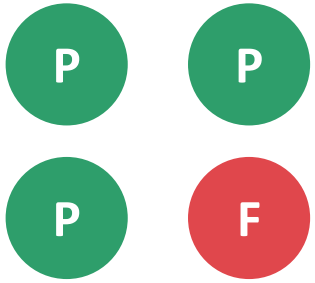
A group is pure when everyone in it has the same result.



Pure

Gini = 0

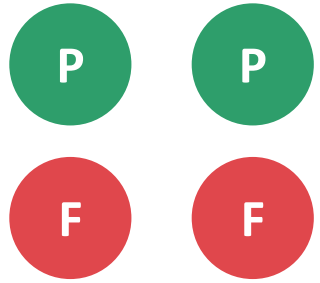
Perfect. No more questions needed.



Mostly one result

Gini = 0.375

Leaning Pass, but not certain.



Completely mixed

Gini = 0.5

A coin toss. The most mixed possible.

Gini impurity: a messiness score

Lower is better. 0 means pure; 0.5 is the most mixed a two-class group can be.

$$\text{Gini} = 1 - (\text{share who passed})^2 - (\text{share who failed})^2$$

The intuition

Pick two students from the group at random (putting the first back before picking again). Gini is the chance that their results are different.



Careful

Gini is not the percentage of wrong predictions. For a group of 7 Pass and 5 Fail, guessing “Pass” is wrong 5/12 = 42% of the time, but its Gini is 0.486.

Worked example: Gini before any question

All 12 students in one group: 7 passed, 5 failed.



1. Share who passed = 7/12 = 0.583, squared = 0.340
2. Share who failed = 5/12 = 0.417, squared = 0.174
3. **Gini = 1 – 0.340 – 0.174 = 0.486**

0.486

Quite mixed (the maximum is 0.5)

We'll call this the parent Gini.

Worked example: Gini of each side

After asking Hours ≤ 4.5? we have two smaller groups. Same formula on each.

YES side: 7 students



Passed 2/7 = 0.286, squared = 0.082

Failed 5/7 = 0.714, squared = 0.510

Gini = 1 − 0.082 − 0.510 = 0.408

NO side: 5 students



Passed 5/5 = 1, squared = 1

Failed 0/5 = 0, squared = 0

Gini = 1 − 1 − 0 = 0 (pure)

Combine the two sides fairly

The YES side holds 7 students and the NO side only 5, so YES should count for more.

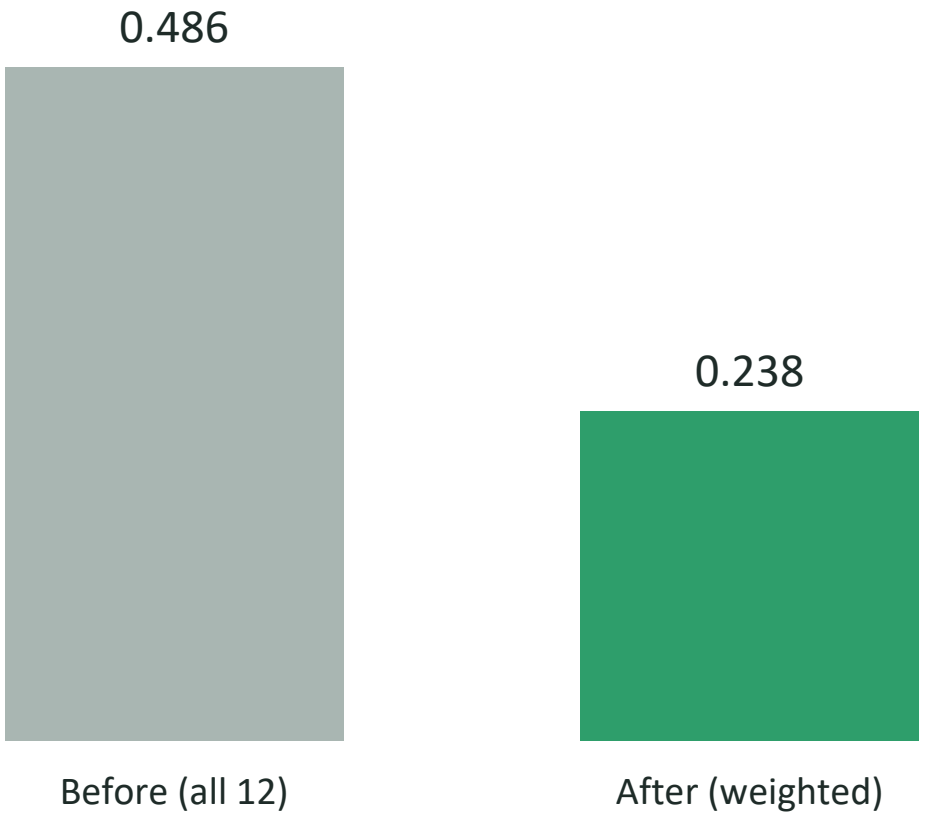
YES: 7 of 12 students, Gini 0.408	NO: 5 of 12, Gini 0

Weighted Gini = 7/12 × 0.408 + 5/12 × 0 = 0.238

Why weight? Think of an average mark across two class sections. A section of 70 students should move the average far more than a section of 5. Each side's Gini counts in proportion to how many students it holds.

How much did the question help?

Compare the messiness before and after asking it.



0.248

Gini gain = 0.486 – 0.238

The bigger the drop, the better the question. At every step the tree looks for the question with the biggest gain, which is the same as the lowest weighted Gini.

Your turn: score Mock ≤ 60?

Three minutes. Same steps as before. Is it better than Hours ≤ 4.5?

YES (mock ≤ 60): 5 students

F F F F P

1 passed, 4 failed

NO (mock > 60): 7 students

F P P P P P P

6 passed, 1 failed

1. Gini of YES side = _____
2. Gini of NO side = _____
3. Weighted = 5/12 × ___ + 7/12 × ___ = ___
4. Gain = 0.486 − _____ = _____

Answer: Mock ≤ 60? scores 0.276

A useful question, but not as good as Hours ≤ 4.5?

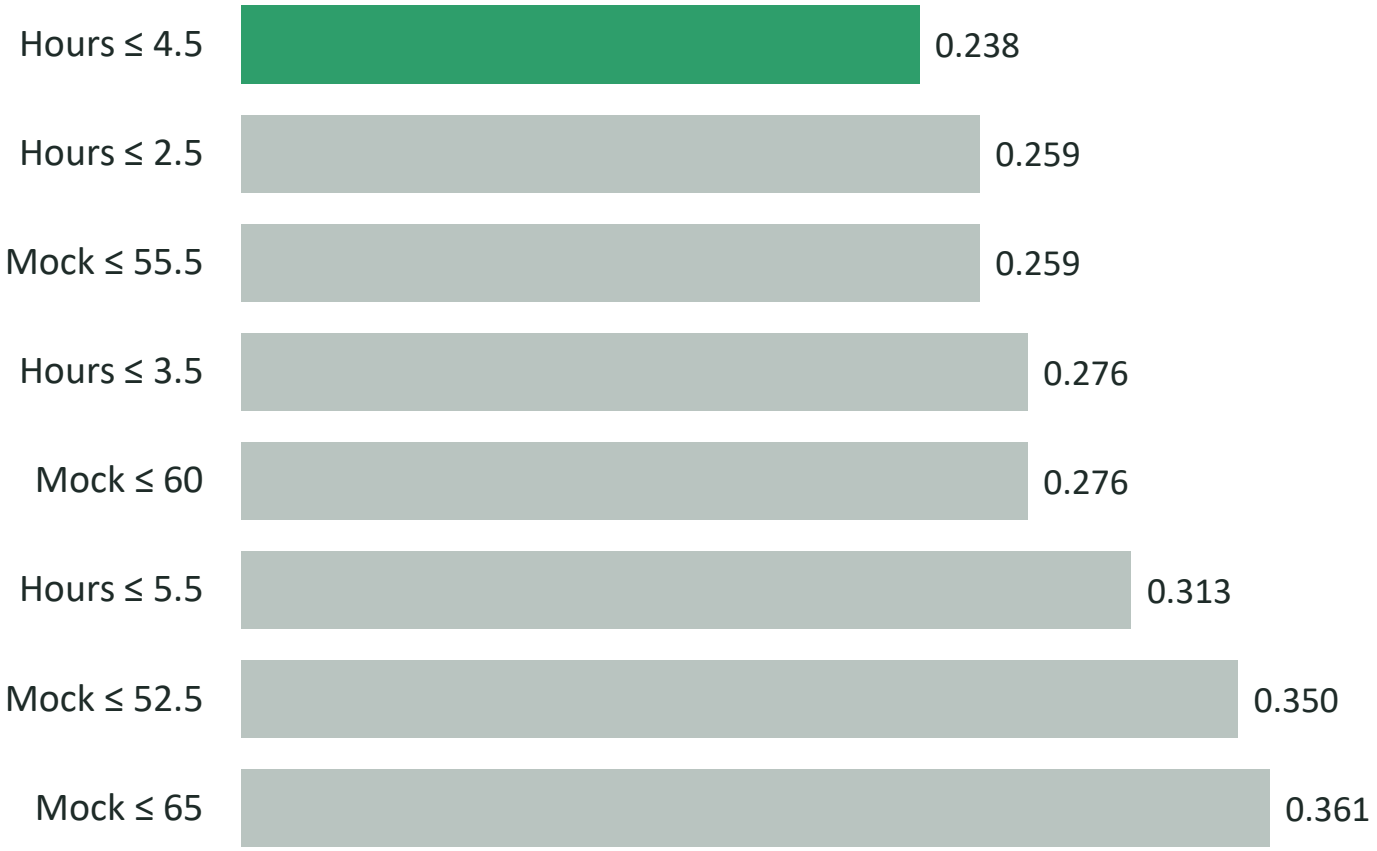
Step	Working
YES side	$1 - (1/5)^2 - (4/5)^2 = 1 - 0.04 - 0.64 = 0.320$
NO side	$1 - (6/7)^2 - (1/7)^2 = 1 - 0.735 - 0.020 = 0.245$
Weighted	$5/12 \times 0.320 + 7/12 \times 0.245 = 0.133 + 0.143 = 0.276$
Gain	**0.486 − 0.276 = 0.210**

Hours ≤ 4.5?
gain 0.248

Mock ≤ 60?
gain 0.210

The tree scores all 18 questions

Here are the top eight by weighted Gini. Lowest wins.



Winner

Hours ≤ 4.5?

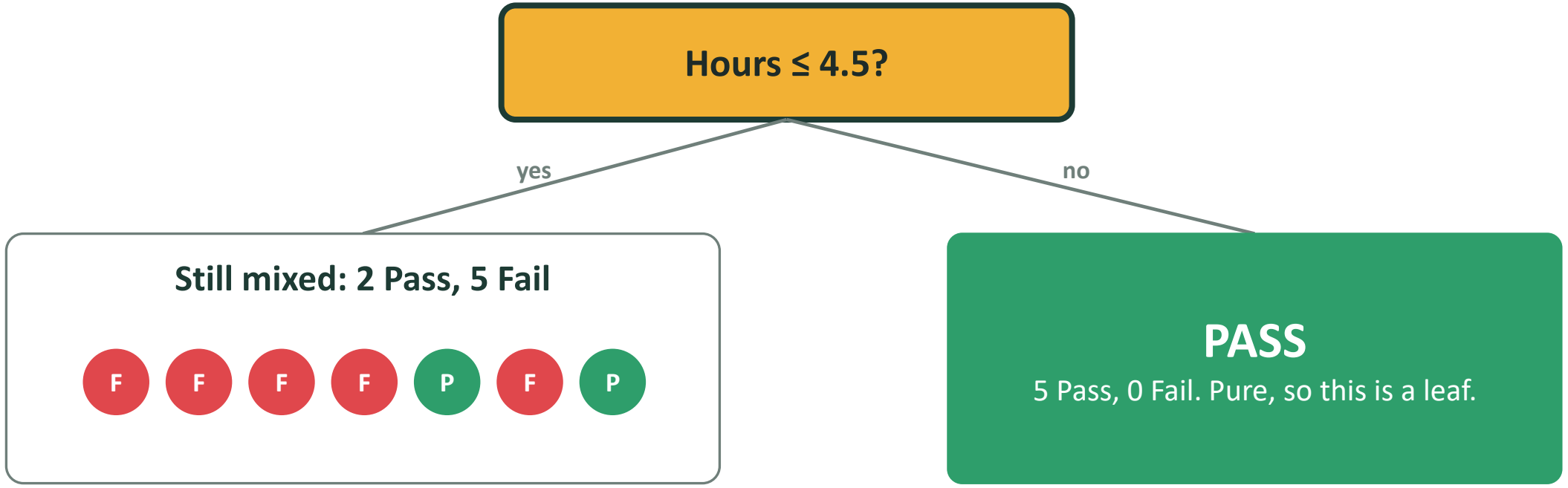
Weighted Gini 0.238, gain 0.248

It becomes the first question: the root of the tree.

Did your poll question make the list?

Question 1 is decided

The data chose Hours ≤ 4.5? as the root. One side is finished; the other is not.



R Riya studied 4 hours, so she goes left into the mixed group. We can't answer her yet. What should the tree do next?

Stop 3 of 5

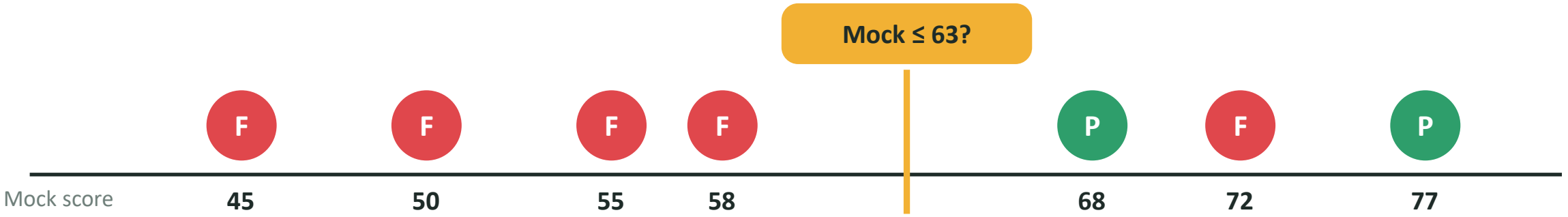
How does the tree build itself?

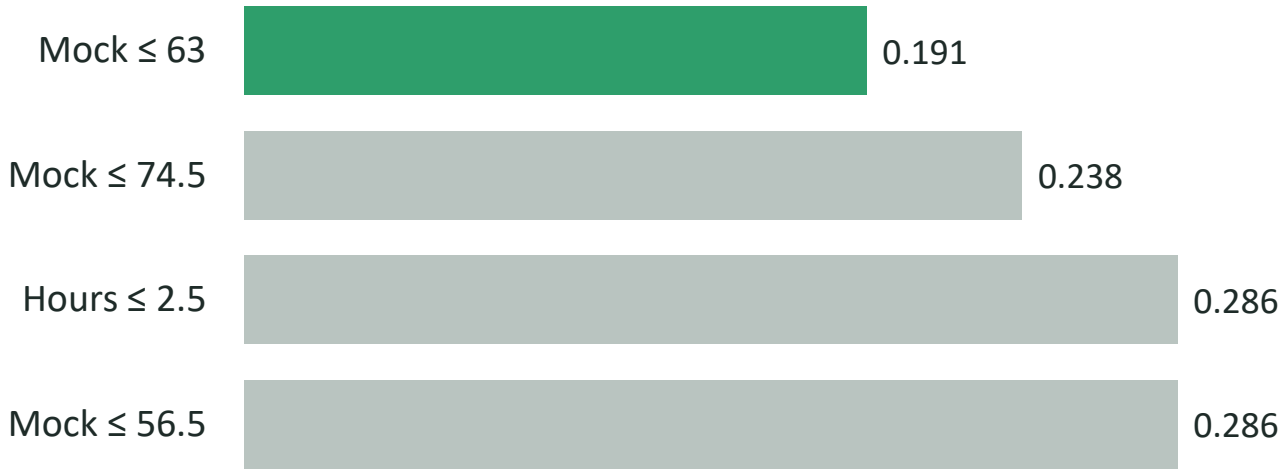
3

Repeat the same recipe inside every mixed group. Then use the finished tree to answer Riya.

Same recipe, smaller group

Only the 7 students on the YES side matter now. Their Gini, 0.408, is the new “before” number.





Mock ≤ 63? wins inside this group.

Gains are now measured from 0.408, this group’s own Gini, not from the original 0.486.

The idea · Measure · **Grow** · Stop · Code

Worked example: scoring Mock ≤ 63?

Same three steps, inside the 7-student group.

YES (mock ≤ 63): 4 students

F F F F

0 Pass, 4 Fail, so Gini = 0 (pure)

NO (mock > 63): 3 students

F P P

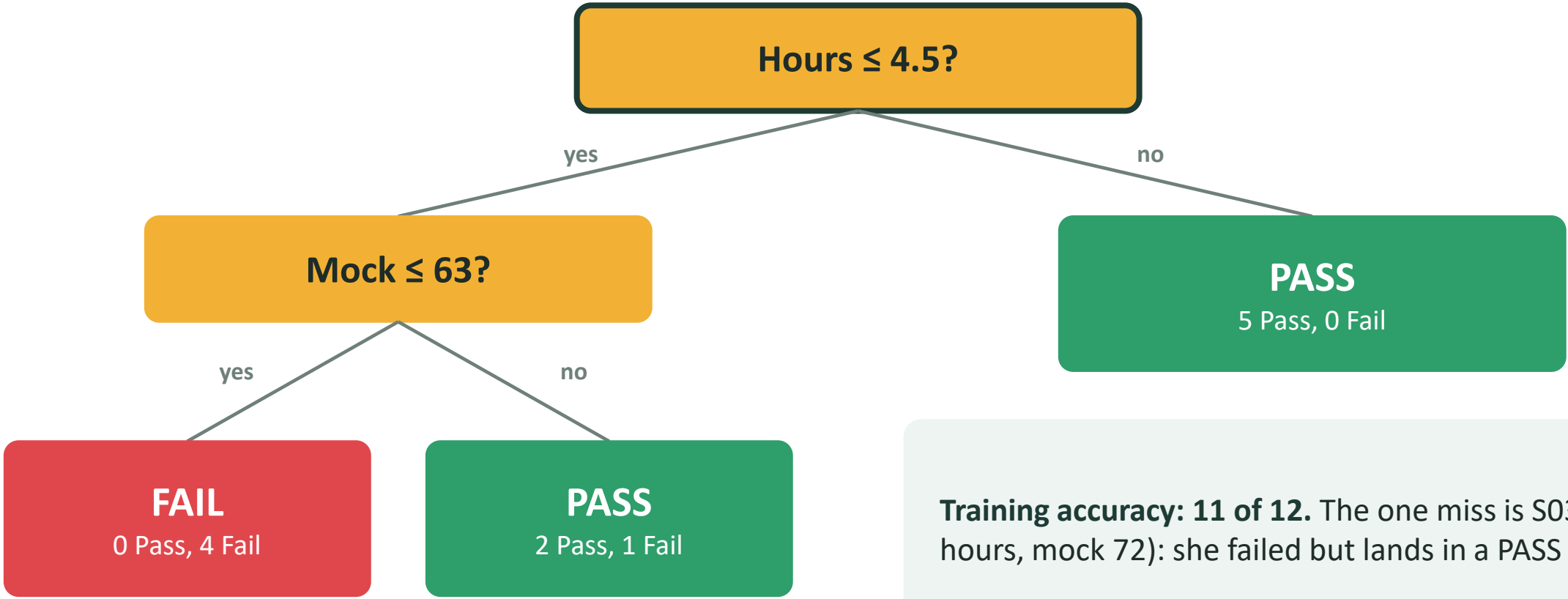
Gini = $1 - (2/3)^2 - (1/3)^2 = 0.444$

Weighted Gini = 4/7 × 0 + 3/7 × 0.444 = 0.190

Gain = 0.408 − 0.190 = 0.218. The best question available for this group.

The finished tree (depth 2)

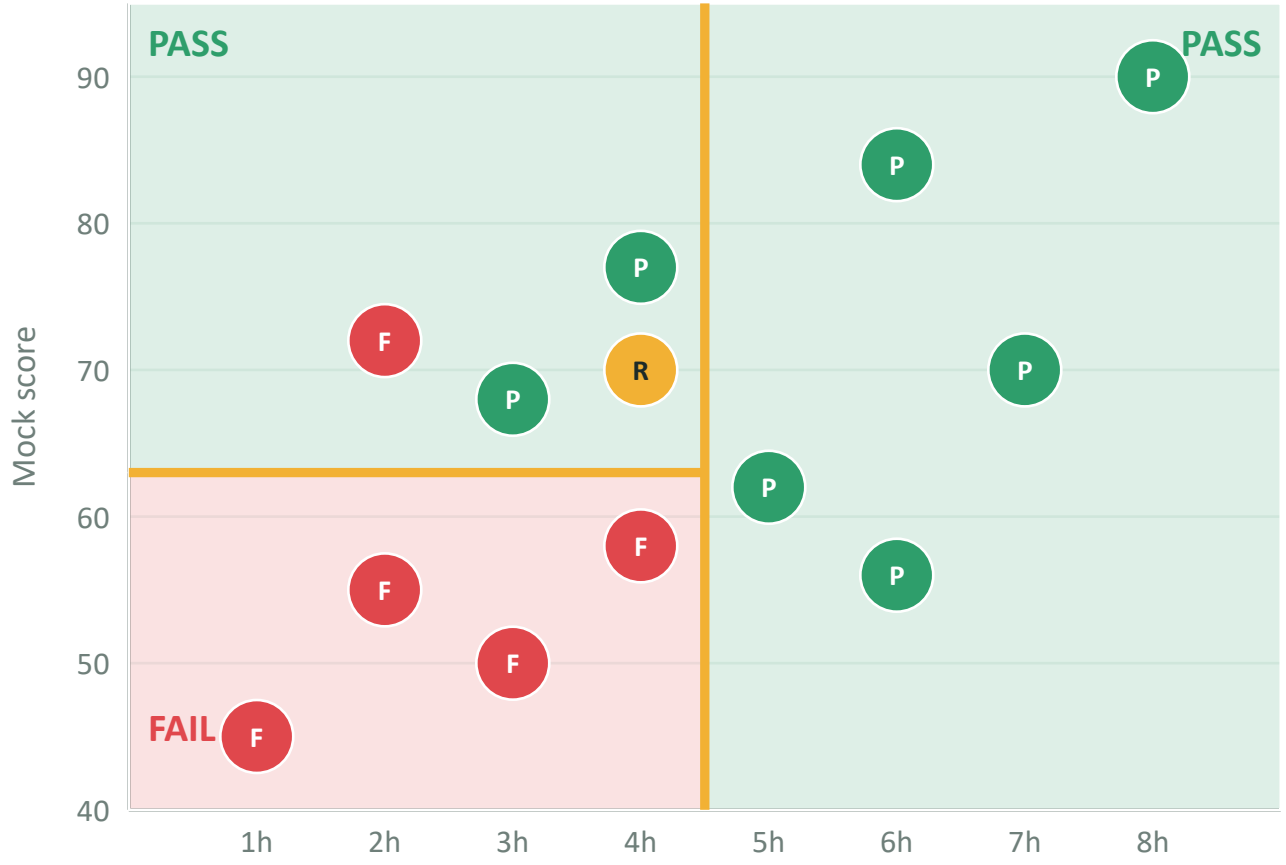
Two questions deep. Every student follows one path from the top to a leaf.



Training accuracy: 11 of 12. The one miss is S03 (2 hours, mock 72): she failed but lands in a PASS leaf.

What the tree does to the data

Each question is a straight cut. Together, the cuts divide the chart into boxes.



Cut 1: Hours ≤ 4.5 (the vertical line).

Cut 2: Mock ≤ 63, only on the left side (the horizontal line).

Riya (R) lands in a PASS box.

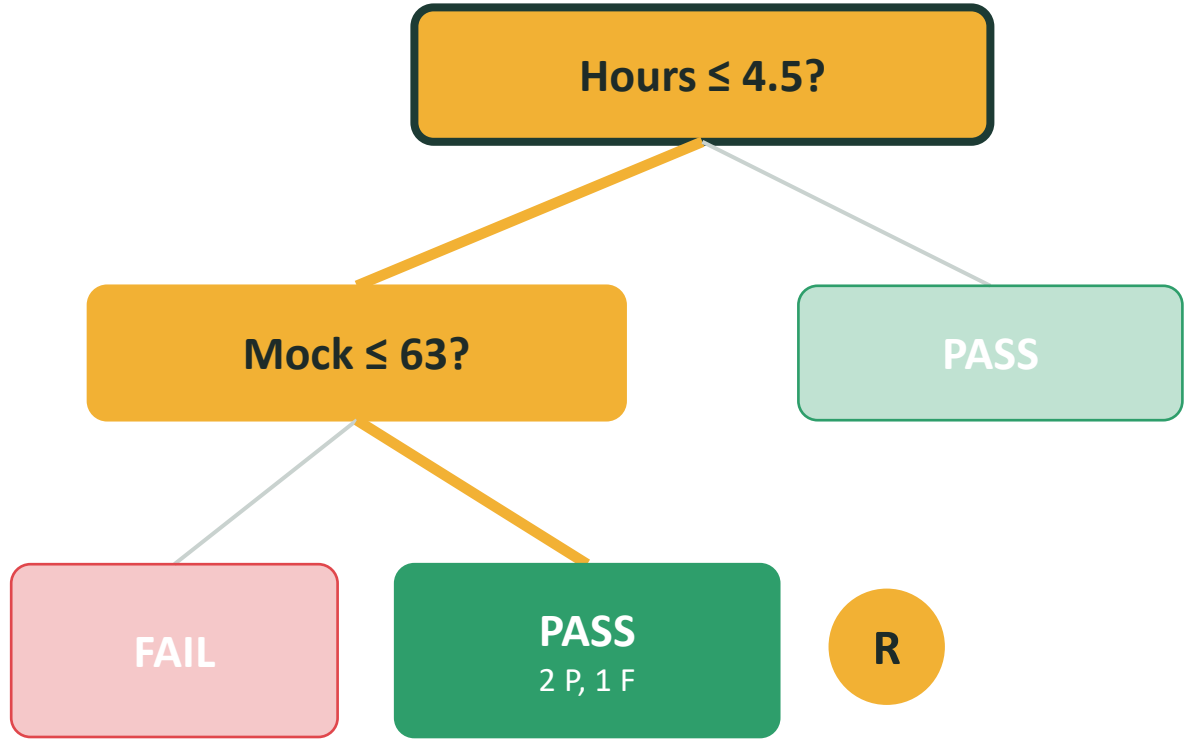
Compare with KNN: its boundary curves around neighbours. A tree's boundary is always made of straight, axis-aligned steps.

So, will Riya pass?

Predicting just means answering the learned questions. No Gini needed.

1. Is 4 hours ≤ 4.5? Yes, so go left
2. Is a mock score of 70 ≤ 63? No, so go right
3. **Leaf: 2 Pass, 1 Fail, so predict PASS**

2 of the 3 training students in that leaf passed, so predict_proba gives Riya a 67% chance of passing.



Your turn: trace three students

45 seconds. Use the finished tree.

Student A

Hours: 2

Mock score: 50

Prediction: _____

Student B

Hours: 5

Mock score: 48

Prediction: _____

Student C

Hours: 3

Mock score: 70

Prediction: _____

The tree: Hours ≤ 4.5? If no, PASS. If yes, ask Mock ≤ 63? If yes, FAIL. If no, PASS.

Answers, and one surprise

The same questions, in the same order, for every student.

Student	Hours ≤ 4.5?	Mock ≤ 63?	Prediction
A (2 h, 50)	Yes	Yes	FAIL
B (5 h, 48)	No	*not asked*	PASS
C (3 h, 70)	Yes	No	PASS

A tree can be confidently wrong when it learned from very little data. That's what the next stop is about.

Surprised by B?

A mock score of 48, yet predicted to pass. Our 12 students never included a hard-working student with a low mock score, so the tree never learned to worry about one.

Stop 4 of 5

When should the tree stop asking?

4

A tree can keep splitting until every training student sits in a pure leaf. That’s usually a bad idea.

Deeper trees fit training data better

Training accuracy on our 12 students as the tree grows.

Depth 1

83%

10 of 12 correct

Only Hours ≤ 4.5?

Depth 2

92%

11 of 12 correct

Adds Mock ≤ 63? on the left side

Depth 3

100%

12 of 12 correct

Adds Hours ≤ 2.5? just to catch one student, S03

100% sounds perfect. But is “2 hours or less with a mock above 63 means Fail” a real pattern, or one unlucky student?

Memorising is not learning

The goal is good predictions for new students, not a perfect score on old ones.

Too shallow

Underfitting

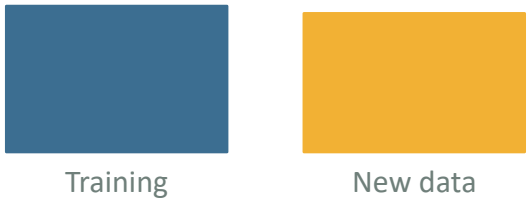


Like a student who only read the chapter titles.

Misses real patterns. Weak on training data and on new data.

Just right

A good balance

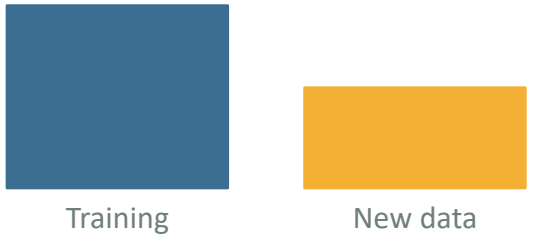


Understands the concepts.

Captures the real pattern. Does well on new data.

Too deep

Overfitting



Memorised last year's answer key.

Perfect on training data, stumbles on new questions.

Brakes that stop a tree growing too far

Settings (hyperparameters) you pass to DecisionTreeClassifier(...).

Setting	In plain English	Tighter setting means
`max_depth`	The most questions any student can be asked	A shorter, simpler tree
`min_samples_split`	A group needs at least this many students before it may be split	Small groups stay as leaves
`min_samples_leaf`	Every leaf must keep at least this many students	No tiny one-student leaves
`max_leaf_nodes`	The most leaves allowed in total	Fewer final answers
`ccp_alpha`	Grow fully, then prune branches that don’t earn their keep	More branches pruned

The first four act while the tree grows (pre-pruning); ccp_alpha acts afterwards (post-pruning). Start by tuning max_depth and min_samples_leaf.

No scaling needed, unlike KNN

Trees compare one feature at a time with a threshold. They never measure distance.

KNN

Measures the distance between students. If one feature is salary in rupees and another is age in years, salary swamps the distance. Scaling is essential.

Decision tree

Asks “Hours ≤ 4.5?”. Convert hours to minutes and it simply asks “Minutes ≤ 270?”. Same students on each side, same tree.

Still essential: turn text categories into numbers, handle missing values, and keep the test set untouched. Not caring about scale is not the same as not caring about data quality.

Stop 5 of 5

The same thing, in scikit-learn

5

First we check our hand-built tree. Then we train on a bigger dataset and tune it properly.

The idea · Measure · Grow · Stop · **Code**

Did we get it right?

Give scikit-learn the same 12 students and compare.

```
from sklearn.tree import DecisionTreeClassifier, export_text

X = [[1,45],[2,55],[2,72],[3,50],[3,68],[4,58],
     [4,77],[5,62],[6,56],[6,84],[7,70],[8,90]]
y = [0,0,0,0,1,0,1,1,1,1,1,1]    # 1 = Pass

tree = DecisionTreeClassifier(max_depth=2)
tree.fit(X, y)
print(export_text(tree, feature_names=["hours", "mock"]))

print(tree.predict([[4, 70]]))          # Riya
print(tree.predict_proba([[4, 70]]))
```

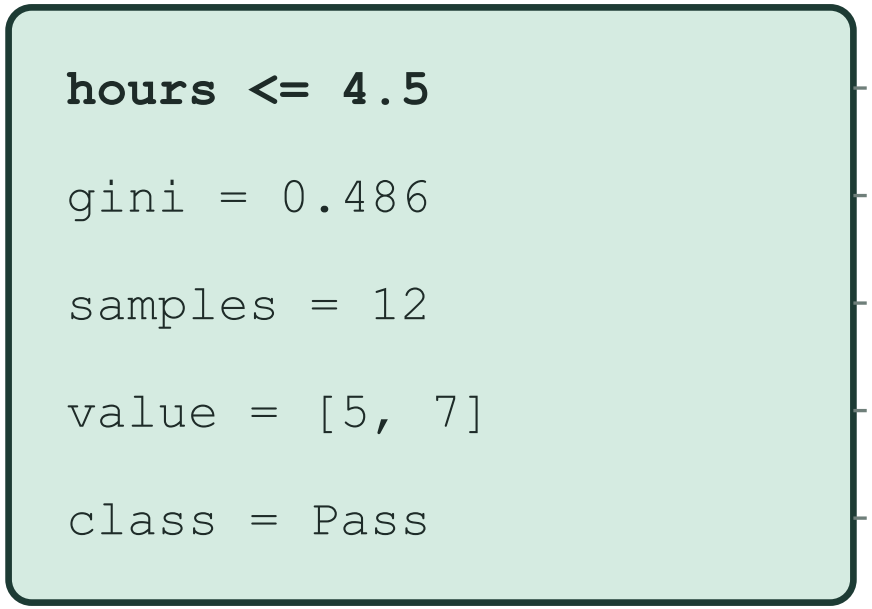
Output

```
|--- hours <= 4.50
|   |--- mock <= 63.00
|   |   |--- class: 0
|   |--- mock >  63.00
|   |   |--- class: 1
|--- hours >  4.50
|   |--- class: 1
[1]
[[0.333 0.667]]
```

Same questions, same thresholds, same answer for Riya.

Reading a scikit-learn tree box

plot_tree draws boxes like this. Every line is something you already calculated.



- `hours <= 4.5` — The question. True goes left, false goes right.
- `gini = 0.486` — The messiness score from our worked example.
- `samples = 12` — How many training students reached this box.
- `value = [5, 7]` — Students per class, in the order [Fail, Pass].
- `class = Pass` — The majority answer at this box.

```
plot_tree(tree, feature_names=["hours", "mock"], class_names=["Fail", "Pass"], filled=True)
```

Now a realistic dataset: 240 students

Same columns, many more rows, so we can hold back a fair test set.

Column	Role
`hours`	Input feature
`mock_score`	Input feature
`passed`	Target: 0 = Fail, 1 = Pass
`student_id`	Just an ID. Leave it out of X

240
students

180
to train

60
to test

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

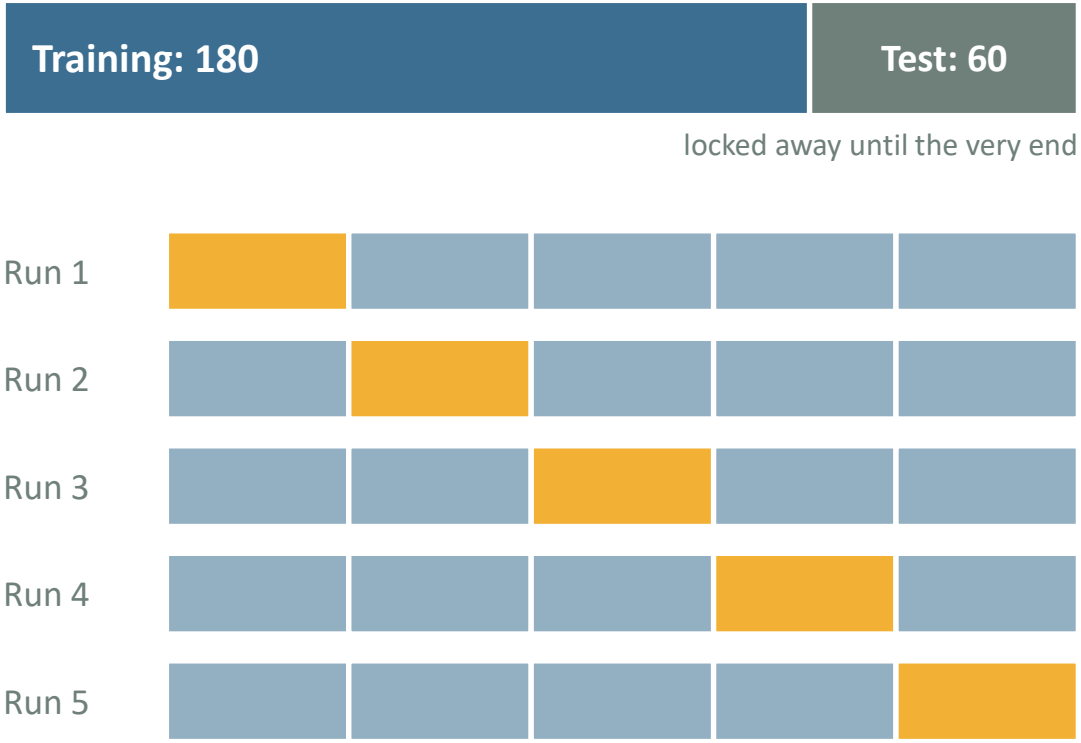
df = pd.read_csv("Decision_Tree_Student_Success_Dataset.csv")
X = df[["hours", "mock_score"]]
y = df["passed"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)      # learn the questions
y_pred = model.predict(X_test)   # answer them
```

Pick the depth with cross-validation

Never tune on the test set. Rotate a validation slice inside the training data instead.



Amber = the 36 students used to score this run. Each candidate setting gets 5 scores; we average them.

```
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True,
                     random_state=42)
params = {
    "criterion": ["gini", "entropy"],
    "max_depth": [2, 3, 4, 5, None],
    "min_samples_leaf": [1, 3, 6, 10],
}
grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    params, cv=cv, scoring="accuracy")
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
```

Tuned, then tested once

Results on the 240-student dataset (random_state = 42).

4

Best max_depth

10

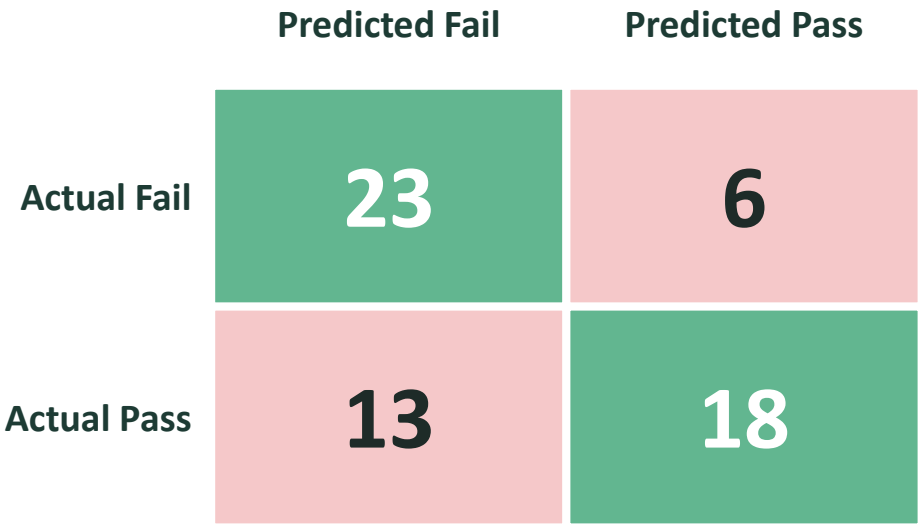
Best min_samples_leaf

gini

Best criterion

74.4%

Mean CV accuracy (baseline at depth 3: 68.9%)



Test accuracy: 41 of 60 = 68.3%

Lower than the CV score, and that's normal. The test set is a fresh, honest check that played no part in choosing the settings.

The whole algorithm on one slide

One loop builds the tree. One walk down the tree makes a prediction.

1. Start with labelled examples
2. List every candidate cut
3. Score each one with weighted Gini
4. Keep the best and split. Repeat in each mixed group
5. Stop at a limit: depth, leaf size
6. **Predict by walking root to leaf**

`fit()` `predict()`

The best trees aren’t the deepest. They’re the ones that work on new students.

Where trees fit in your toolkit

What each model learns during fit(), and what it does during predict().

Model	fit() learns	predict() does
Linear regression	A slope and an intercept	Plugs the inputs into the line
Logistic regression	Coefficients for a weighted score	Score, then sigmoid, then class
KNN	Almost nothing: it stores the training data	Finds the nearest neighbours and lets them vote
Decision tree	**Questions: which feature, which threshold, what order**	**Walks from the root to a leaf**

Exit ticket

Answer without looking at the slides or the code.

1. Where do the candidate thresholds come from?
2. Why do we weight each side’s Gini by its size?
3. Your tree scores 100% on training data. Why might that be bad news?
4. Walk through how the tree predicted Riya’s result.

Coming next

Random Forest

One tree is easily swayed by a few students, like S03. What if we grew hundreds of slightly different trees and let them vote?